

REST Usage Governance and Metering Boundaries

Version 1.4.2-SNAPSHOT, 2026-09-21T22:14:39Z

Table of Contents

1. Runtime modes	2
2. Stable identity	3
3. Property manifest	4
4. Extension contracts	5
5. Storage and scaling boundary	6
6. Network guard boundary	7

Quantum REST usage governance is opt-in and defaults to **OFF**. It provides automatic post-authentication endpoint admission, a bounded per-JVM token bucket, and an OSS observation SPI. It does not use a MongoDB request counter, does not poll, and does not enable a pre-authentication network limiter.

Chapter 1. Runtime modes

OFF

The default. The filter performs no policy lookup, bucket allocation, or observation.

OBSERVE

The filter resolves the same trusted endpoint and principal identity used by enforcement, consumes the local bucket, and emits an observation. Exhaustion is recorded as `WOULD_REJECT`; the request continues. Unavailable enforcement state is recorded as `ENFORCEMENT_ERROR`; the request continues. Registry capacity pressure is recorded as `CAPACITY_BYPASSED`; the request continues without allocating an unbounded key.

ENFORCE

Exhaustion returns HTTP 429 with the typed `QuantumUsageGovernanceError` body and a whole-second `Retry-After` header. Missing endpoint identity, trusted principal identity, or policy state returns a typed HTTP 503 and does not invoke the endpoint. Bucket-registry capacity is never allowed to become a cross-tenant 503 primitive.

Chapter 2. Stable identity

The policy key is:

```
policy id/version + functional area/domain/OpenAPI operationId + tenant + subject
```

The area and domain come from `@FunctionalMapping`, including model mappings made available by the security runtime. The operation identity comes from an explicit `@Operation(operationId = "...")`; the stable JAX-RS method name is the runtime fallback and requires the matching `SmallRye METHOD` OpenAPI operation-ID strategy. Mutable URL paths are not policy identity. When governance is active, OpenAPI generation validates that operation IDs are present and unique and that exact configured selectors refer to generated operations.

Tenant and subject are read only from the effective framework `PrincipalContext` established by `SecurityFilter`. The tenant is the effective data-domain tenant after validated realm override; the subject is the provider subject with framework user ID as fallback. Raw identity attributes, headers, query parameters, and payload fields are not identity authorities. Endpoints explicitly annotated `@PermitAll` are outside this post-auth layer and need separately managed edge/network controls where appropriate. A class-level `@PermitAll` does not exempt a method that declares `@Authenticated`, `@RolesAllowed`, or `@DenyAll`.

Chapter 3. Property manifest

Property	Default	Description
<code>quantum.rest.usage.mode</code>	<code>OFF</code>	<code>OFF</code> , <code>OBSERVE</code> , or <code>ENFORCE</code> .
<code>quantum.rest.usage.policy.id</code>	<code>default-rest-usage</code>	Stable policy identifier.
<code>quantum.rest.usage.policy.version</code>	<code>1</code>	Policy version included in bucket identity.
<code>quantum.rest.usage.policy.request-limit</code>	<code>1000</code>	Bucket capacity and refill token count.
<code>quantum.rest.usage.policy.refill-period</code>	<code>PT5S</code>	Positive ISO-8601 refill period.
<code>quantum.rest.usage.policy.endpoints</code>	<code>*</code>	Comma-separated <code>area:domain:operationId</code> selectors or <code>*</code> .
<code>quantum.rest.usage.allow-unmatched-endpoints</code>	<code>true</code>	When <code>ENFORCE</code> resolves a trusted endpoint but the policy source returns no matching policy, admit it as <code>BYPASSED</code> . Set to <code>false</code> to fail closed with the typed <code>USAGE_POLICY_NOT_FOUND</code> 503. This flag never covers indeterminate endpoint, identity, source, or bucket state; those always fail closed.
<code>quantum.rest.usage.bucket.max-tracked-keys</code>	<code>10000</code>	Strict per-JVM bound for policy/endpoint/tenant/subject keys.
<code>quantum.rest.usage.bucket.idle-timeout</code>	<code>PT10M</code>	Positive ISO-8601 idle period before request-triggered eviction.
<code>mp.openapi.extensions.smallrye.operationIdStrategy</code>	<code>none</code>	Must be explicitly set to <code>METHOD</code> whenever governance is active so generated OpenAPI and runtime operation identities cannot diverge.

Invalid active configuration stops startup. `OFF` deliberately leaves dormant policy values unparsed so an upgrade remains inert. A typical observation-first rollout is:

```
quantum.rest.usage.mode=OBSERVE
mp.openapi.extensions.smallrye.operationIdStrategy=METHOD
quantum.rest.usage.policy.id=standard-api
quantum.rest.usage.policy.version=2026-08-15
quantum.rest.usage.policy.request-limit=100
quantum.rest.usage.policy.refill-period=PT1M
quantum.rest.usage.policy.endpoints=sales:order:listOrders,sales:order:createOrder
quantum.rest.usage.allow-unmatched-endpoints=true
```

Chapter 4. Extension contracts

The reusable policy, admission, observation, identity, and error contracts live in `quantum-contract-core`. This is the acyclic leaf: `quantum-rest-core` was rejected because it depends on `quantum-morphia-repos`, and `quantum-models` was rejected because it directly carries Morphia persistence types.

`UsagePolicySource` is the policy lookup SPI. Its default CDI implementation is `PropertyUsagePolicySource`; applications can provide another implementation without changing the filter. A source receives `UsageRequest` and returns `Optional<UsagePolicy>`. Returning an inapplicable policy, returning `null`, or throwing is an explicit policy-state failure.

`UsageObserver` is the downstream observation SPI. `UsageObservation` contains:

- endpoint, tenant, subject, and HTTP method;
- response status and latency;
- request and response byte counts (`-1` when unknown); and
- `ADMITTED`, `WOULD_REJECT`, `REJECTED`, `BYPASSED`, `CAPACITY_BYPASSED`, or `ENFORCEMENT_ERROR`, plus policy/retry/error details in the admission result.

Observers run after a response exists. Tenant and subject identifiers are sensitive; trusted observer implementations must not log them unredacted or export them to untrusted sinks. An observer can bridge to metrics, events, or durable billing, but its failure cannot retroactively change admission. Keep exporters bounded and nonblocking.

Chapter 5. Storage and scaling boundary

The admission bucket is a framework-owned atomic in-memory token bucket with an injectable monotonic clock. The registry is strictly bounded and evicts idle entries opportunistically during requests; there is no polling, scheduler, or third-party bucket runtime. When the bound is reached, a new key is admitted without a bucket and observed as `CAPACITY_BYPASSED`; this preserves the memory bound without allowing one tenant's cardinality to deny another tenant.

Each application replica has its own allowance. Therefore a deployment with three replicas has approximately three times the configured aggregate request capacity. Phase 1 intentionally prefers a deterministic, bounded OSS request path over a MongoDB read/modify/write counter. Cluster-wide entitlement and durable billing can consume the OSS contracts in a later, separately reviewed integration.

The existing `UsageMeteringService` and persistent token-allocation models are manual durable metering APIs. They are not called by `UsageGovernanceFilter` and must not be inserted into its admission path. If used, connect them downstream through an observer or event boundary with explicit delivery and backpressure semantics.

Chapter 6. Network guard boundary

Post-auth usage governance does not protect public authentication work. Deployments that need source-network denial-of-service protection should configure an edge or HTTP-layer guard separately. Such a guard uses a network identity and must never be described as tenant, subject, quota, or billing enforcement.